

《数据科学》实验报告

班级	003	姓名	史润	学号	20241220
实验题目	Python 数据科学工具包的实践				
实验时间	2025-9-27	实验地点	DS1408		
学年学期	2025-2026(1)	实验性质	☐ 验证性 ☐ 设计性 ☒ 综合性		

实验过程和结果

1. 创建一个一维数组 a 初始化为[2, 4, 6, 8, 10, 12]。打印并输出如下内容
代码：

```
import numpy as np
# 1. 创建一维数组 a
a = np.array([2, 4, 6, 8, 10, 12])
print("原始数组 a:", a)
# (1) a 的类型
print("\n(1) a 的类型:", type(a))
# (2) a 的 shape 和 size
print("\n(2) a 的 shape:", a.shape)
print("    a 的 size:", a.size)
# (3) 输出 a 的第 5 个元素
print("\n(3) a 的第 5 个元素:", a[4])
# (4) 将 a 转换为 2*3 的矩阵，并输出新矩阵的 shape 和 size
matrix = a.reshape(2, 3)
print("\n(4) 转换后的 2*3 矩阵:\n", matrix)
print("    新矩阵的 shape:", matrix.shape)
print("    新矩阵的 size:", matrix.size)
# (5) 从新矩阵中索引输出[10, 12]
result = matrix[1, 1:] # 第二行的后两个元素
print("\n(5) 索引输出结果:", result)
# (6) 利用"布尔值索引"输出 a 中大于 4 的数
greater_than_4 = a[a > 4]
print("\n(6) a 中大于 4 的数:", greater_than_4)
# (7) 重新指定数组 a 的数据类型为浮点数（float64），并输出其 dtype
a_float = a.astype(np.float64)
print("\n(7) 转换为 float64 后的 dtype:", a_float.dtype)
# (8) 将 a 中全部元素缩小 10，对比 around、ceil、floor 的不同舍入效果
a_scaled = a_float / 10
print("\n(8) 缩小 10 倍后的数组:", a_scaled)
print("    around 舍入结果:", np.around(a_scaled))
```

```
print("    ceil 舍入结果:", np.ceil(a_scaled))
print("    floor 舍入结果:", np.floor(a_scaled))
```

结果:

```
原始数组a: [ 2  4  6  8 10 12]

(1) a的类型: <class 'numpy.ndarray'>

(2) a的shape: (6,)
    a的size: 6

(3) a的第5个元素: 10

(4) 转换后的2*3矩阵:
    [[ 2  4  6]
     [ 8 10 12]]
    新矩阵的shape: (2, 3)
    新矩阵的size: 6

(5) 索引输出结果: [10 12]

(6) a中大于4的数: [ 6  8 10 12]

(7) 转换为float64后的dtype: float64

(8) 缩小10倍后的数组: [0.2 0.4 0.6 0.8 1.  1.2]
    around舍入结果: [0. 0. 1. 1. 1. 1.]
    ceil舍入结果: [1. 1. 1. 1. 1. 2.]
    floor舍入结果: [0. 0. 0. 0. 1. 1.]
```

2. 练习 numpy 中以下函数的用法

代码:

```
import numpy as np
```

(1) empty: 创建指定形状和类型的未初始化数组

```
empty_arr = np.empty((2, 3))
print("\n(1) empty 创建的 2x3 数组: ")
print(empty_arr)
```

(2) zeros: 创建全为 0 的数组

```
zeros_arr = np.zeros((3, 2), dtype=int)
print("\n(2) zeros 创建的 3x2 整数数组: ")
print(zeros_arr)
```

(3) ones: 创建全为 1 的数组

```
ones_arr = np.ones((2, 2, 2), dtype=float)
print("\n(3) ones 创建的 2x2x2 浮点数组: ")
print(ones_arr)
```

(4) full: 创建填充指定值的数组

```
full_arr = np.full((3, 4), 7)
print("\n(4) full 创建的 3x4 填充 7 的数组: ")
print(full_arr)
```

(5) arange: 创建[1, 2, 3, 4, 5]

```
arr1 = np.arange(1, 6) # 从 1 开始, 到 6 结束 (不包含 6)
```

```

print("\n(5) arange 创建的 arr1: ")
print(arr1)

# (6) linspace: 创建[1, 3, 5, 7, 9]
# 从 1 到 9, 生成 5 个等间隔的数
arr2 = np.linspace(1, 9, 5, dtype=int)
print("\n(6) linspace 创建的 arr2: ")
print(arr2)

# (7) rand: 生成 0-1 均匀分布的 3×4×2 随机三维数组
# 设置随机种子, 保证结果可复现
np.random.seed(42)
rand_3d = np.random.rand(3, 4, 2)
print("\n(7) rand 生成的 3×4×2 随机三维数组: ")
print(rand_3d)
print("数组形状: ", rand_3d.shape)

```

结果:

<pre> (1) empty创建的2×3数组: [[0. 0. 0.] [0. 1. 1.]] (2) zeros创建的3×2整数数组: [[0 0] [0 0] [0 0]] (3) ones创建的2×2×2浮点数组: [[[1. 1.] [1. 1.]] [[1. 1.] [1. 1.]]] (4) full创建的3×4填充7的数组: [[7 7 7 7] [7 7 7 7] [7 7 7 7]] (5) arange创建的arr1: [1 2 3 4 5] </pre>	<pre> (6) linspace创建的arr2: [1 3 5 7 9] (7) rand生成的3×4×2随机三维数组: [[[0.37454012 0.95071431] [0.73199394 0.59865848] [0.15601864 0.15599452] [0.05808361 0.86617615]] [[0.60111501 0.70807258] [0.02058449 0.96990985] [0.83244264 0.21233911] [0.18182497 0.18340451]] [[0.30424224 0.52475643] [0.43194502 0.29122914] [0.61185289 0.13949386] [0.29214465 0.36636184]]] 数组形状: (3, 4, 2) </pre>
---	---

3. 创建两个 3 行 3 列的数组, 分别定义为 A 和 B, 其元素值任意设定。对两个数组分别实现下列内容

代码:

```

import numpy as np

# 创建两个 3 行 3 列的数组 A 和 B
np.random.seed(42) # 设置随机种子, 确保结果可复现
A = np.random.randint(1, 10, size=(3, 3)) # 1-9 的随机整数
B = np.random.randint(1, 10, size=(3, 3)) # 1-9 的随机整数
print("数组 A:")
print(A)
print("\n数组 B:")
print(B)

```

```
# (1) 加法、减法、乘法（矩阵乘法和哈达玛积）运算
print("\n(1) 数组运算结果: ")
print("A + B:")
print(A + B)
print("\nA - B:")
print(A - B)
print("\n哈达玛积 (A * B - 对应元素相乘):")
print(A * B)
print("\n矩阵乘法 (A @ B):")
print(A @ B) # 等同于 np.matmul(A, B)

# (2) 比较运算，输出 A 大于等于 B 的比较结果
print("\n(2) A >= B 的比较结果:")
print(A >= B)

# (3) 输出 A 中大于等于 B 的值的索引
print("\n(3) A 中大于等于 B 的值的索引:")
indices = np.where(A >= B)
# 使用 zip 组合索引并输出
for idx in zip(indices[0], indices[1]):
    print(f"索引 {idx}: A[{idx}] = {A[idx]}, B[{idx}] = {B[idx]}")

# (4) 输出 A*B 的转置矩阵
print("\n(4) A*B 的转置矩阵:")
product = A * B
print(product.T) # 等同于 np.transpose(product)

# (5) 输出 A 和 B 数值对应元素相除后的余数
print("\n(5) A 和 B 对应元素相除的余数:")
print(A % B)

# (6) 输出 A 第二行的元素（注意：索引从 0 开始，第二行即索引为 1）
print("\n(6) A 第二行的元素:")
print(A[1])

# (7) 输出 A 第三行第二列的元素[2,1]
print("\n(7) A 第三行第二列的元素:")
print(A[2, 1])

# (8) 删除 A 的第一行，并输出
print("\n(8) 删除 A 的第一行后:")
A_without_first_row = np.delete(A, 0, axis=0)
print(A_without_first_row)
```

(9) 删除 B 的第一列，并输出

```
print("\n(9) 删除 B 的第一列后:")
B_without_first_col = np.delete(B, 0, axis=1)
print(B_without_first_col)
```

(10) 通过 np.sum 函数输出对 B 的按列求和，以及 B 的按行求和

```
print("\n(10) B 的求和结果:")
print("按列求和:", np.sum(B, axis=0))
print("按行求和:", np.sum(B, axis=1))
```

结果:

```
数组A:
[[7 4 8]
 [5 7 3]
 [7 8 5]]

数组B:
[[4 8 8]
 [3 6 5]
 [2 8 6]]

(1) 数组运算结果:
A + B:
[[11 12 16]
 [ 8 13  8]
 [ 9 16 11]]

A - B:
[[ 3 -4  0]
 [ 2  1 -2]
 [ 5  0 -1]]

矩阵乘法 (A @ B):
[[ 56 144 124]
 [ 47 106  93]
 [ 62 144 126]]

(2) A >= B 的比较结果:
[[ True False  True]
 [ True  True False]
 [ True  True False]]

(4) A*B的转置矩阵:
[[28 15 14]
 [32 42 64]
 [64 15 30]]

(5) A和B对应元素相除的余数:
[[3 4 0]
 [2 1 3]
 [1 0 5]]

(6) A第二行的元素:
[5 7 3]

(7) A第三行第二列的元素:
8

(8) 删除A的第一行后:
[[5 7 3]
 [7 8 5]]

(9) 删除B的第一列后:
[[8 8]
 [6 5]
 [8 6]]

(10) B的求和结果:
按列求和: [ 9 22 19]
按行求和: [20 14 16]
```

```
(3) A中大于等于B的值的索引:
索引 (np.int64(0), np.int64(0)): A[(np.int64(0), np.int64(0))] = 7, B[(np.int64(0), np.int64(0))] = 4
索引 (np.int64(0), np.int64(2)): A[(np.int64(0), np.int64(2))] = 8, B[(np.int64(0), np.int64(2))] = 8
索引 (np.int64(1), np.int64(0)): A[(np.int64(1), np.int64(0))] = 5, B[(np.int64(1), np.int64(0))] = 3
索引 (np.int64(1), np.int64(1)): A[(np.int64(1), np.int64(1))] = 7, B[(np.int64(1), np.int64(1))] = 6
索引 (np.int64(2), np.int64(0)): A[(np.int64(2), np.int64(0))] = 7, B[(np.int64(2), np.int64(0))] = 2
索引 (np.int64(2), np.int64(1)): A[(np.int64(2), np.int64(1))] = 8, B[(np.int64(2), np.int64(1))] = 8
```

4. 将下面的数据创建为 DataFrame: { "Fruits": ["Apple", "Orange", "Banana", "Mango", "Watermelon", "Grapefruit"], "Price": [3, 2, 2.5, 3.5, 2, 3.5] }, 并完成以下内容

代码:

```
import pandas as pd
import numpy as np
# 创建 DataFrame
```

```
data = {
    "Fruits": [
        "Apple",
        "Orange",
        " Banana ",
        " Mango ",
        " Watermelon ",
        " Grapefruit ",
    ],
    "Price": [3, 2, 2.5, 3.5, 2, 3.5],
}
df = pd.DataFrame(data)

# (1) 打印 DataFrame 的结果
print("(1) 原始 DataFrame: ")
print(df)
print()

# (2) 分别使用 loc 和 iloc 抽出"Orange"一行数据
print("(2) 使用 loc 提取'Orange'行: ")
print(df.loc[df["Fruits"] == "Orange"])
print("\n使用 iloc 提取'Orange'行（已知索引为1）: ")
print(df.iloc[1])
print()

# (3) 提取含有字符串 " Banana "的行，并输出返回值的数据类型
banana_row = df[df["Fruits"] == " Banana "]
print("(3) 含有' Banana '的行: ")
print(banana_row)
print("返回值的数据类型: ", type(banana_row))
print()

# (4) 提取出 Price 最贵的水果所在的行
max_price_row = df[df["Price"] == df["Price"].max()]
print("(4) 价格最贵的水果行: ")
print(max_price_row)
print()

# (5) 提取出价格为 3 元的水果名称
price_3_fruits = df[df["Price"] == 3]["Fruits"]
print("(5) 价格为 3 元的水果名称: ")
print(price_3_fruits.tolist())
print()
```

```
# (6) 添加一行数据['Peach', 5] (使用 pandas.concat)
new_row = pd.DataFrame({"Fruits": ["Peach"], "Price": [5]})
df = pd.concat([df, new_row], ignore_index=True)
print("(6) 添加 Peach 后的 DataFrame: ")
print(df)
print()

# (7) 添加一列数据“销量”，值自拟
df["销量"] = [10, 15, 8, 12, 20, 5, 7]
print("(7) 添加销量列后的 DataFrame: ")
print(df)
print()

# (8) 添加一列数据“总金额”，其值通过代码计算获得 (价格×销量)
df["总金额"] = df["Price"] * df["销量"]
print("(8) 添加总金额列后的 DataFrame: ")
print(df)
print()

# (9) 将表单按照总金额从高到低重新排序
df_sorted = df.sort_values(by="总金额", ascending=False)
print("(9) 按总金额从高到低排序: ")
print(df_sorted)
print()

# (10) 分别使用 loc 和 iloc 抽出 Peach 和 Banana 行的数据
# 先找到对应索引
peach_index = df_sorted[df_sorted["Fruits"] == "Peach"].index[0]
banana_index = df_sorted[df_sorted["Fruits"] == "Banana"].index[0]

print("(10) 使用 loc 提取 Peach 和 Banana 行: ")
print(df_sorted.loc[[peach_index, banana_index]])

# 重置索引以便使用 iloc
df_sorted_reset = df_sorted.reset_index(drop=True)
peach_pos = df_sorted_reset[df_sorted_reset["Fruits"] == "Peach"].index[0]
banana_pos = df_sorted_reset[df_sorted_reset["Fruits"] == "Banana"].index[0]

print("\n使用 iloc 提取 Peach 和 Banana 行: ")
print(df_sorted_reset.iloc[[peach_pos, banana_pos]])
print()

# (11) 提取并输出总金额高于所有总金额中位数的水果的单价
```

```
median_total = df["总金额"].median()
high_value_prices = df[df["总金额"] > median_total]["Price"]
print("(11) 总金额高于中位数的水果单价: ")
print(high_value_prices.tolist())
print()

# (12) 将表单中的列标题全部改为中文
df = df.rename(columns={"Fruits": "水果", "Price": "价格"})
print("(12) 列标题改为中文后的 DataFrame: ")
print(df)
print()

# (13) 将 DataFrame 输出为"Fruits price.csv"文件
df.to_csv("Fruits price.csv", index=False, encoding="utf-8-sig")
print("(13) 'Fruits price.csv'文件已保存到当前目录")
print()

# (14) 读取"Fruits price.xlsx"文件并输出前 5 行数据
# 注意: 需要安装 openpyxl 库才能读取 xlsx 文件 (pip install openpyxl)
try:
    df_excel = pd.read_excel("Fruits price.xlsx")
    print("(14) 读取的 Excel 文件前 5 行数据: ")
    print(df_excel.head(5))
except FileNotFoundError:
    print("(14) 请先在 Excel 中将 csv 文件另存为 xlsx 文件后再运行此部分")
except ImportError:
    print("(14) 请先安装 openpyxl 库 (pip install openpyxl) 以读取 xlsx 文件")

结果:
```

```
(1) 原始DataFrame:
   Fruits  Price
0   Apple   3.0
1  Orange   2.0
2  Banana   2.5
3   Mango   3.5
4 Watermelon 2.0
5 Grapefruit 3.5

(2) 使用loc提取'Orange'行:
   Fruits  Price
1  Orange   2.0

使用iloc提取'Orange'行 (已知索引为1):
   Fruits  Price
1  Orange   2.0
Name: 1, dtype: object

(3) 含有' Banana '的行:
   Fruits  Price
2  Banana   2.5
返回值的数据类型: <class 'pandas.core.frame.DataFrame'>
```

```
(4) 价格最贵的水果行:
   Fruits  Price
3   Mango   3.5
5 Grapefruit 3.5

(5) 价格为3元的水果名称:
['Apple']

(6) 添加Peach后的DataFrame:
   Fruits  Price
0   Apple   3.0
1  Orange   2.0
2  Banana   2.5
3   Mango   3.5
4 Watermelon 2.0
5 Grapefruit 3.5
6   Peach   5.0

(7) 添加销量列后的DataFrame:
   Fruits  Price  销量
0   Apple   3.0   10
1  Orange   2.0   15
2  Banana   2.5    8
3   Mango   3.5   12
4 Watermelon 2.0   20
5 Grapefruit 3.5    5
6   Peach   5.0    7
```


(8) 添加总金额列后的DataFrame:

	Fruits	Price	销量	总金额
0	Apple	3.0	10	30.0
1	Orange	2.0	15	30.0
2	Banana	2.5	8	20.0
3	Mango	3.5	12	42.0
4	Watermelon	2.0	20	40.0
5	Grapefruit	3.5	5	17.5
6	Peach	5.0	7	35.0

(9) 按总金额从高到低排序:

	Fruits	Price	销量	总金额
3	Mango	3.5	12	42.0
4	Watermelon	2.0	20	40.0
6	Peach	5.0	7	35.0
0	Apple	3.0	10	30.0
1	Orange	2.0	15	30.0
2	Banana	2.5	8	20.0
5	Grapefruit	3.5	5	17.5

(10) 使用loc提取Peach和Banana行:

	Fruits	Price	销量	总金额
6	Peach	5.0	7	35.0
2	Banana	2.5	8	20.0

使用iloc提取Peach和Banana行:

	Fruits	Price	销量	总金额
2	Peach	5.0	7	35.0
5	Banana	2.5	8	20.0

(11) 总金额高于中位数的水果单价:

[3.5, 2.0, 5.0]

(12) 列标题改为中文后的DataFrame:

	水果	价格	销量	总金额
0	Apple	3.0	10	30.0
1	Orange	2.0	15	30.0
2	Banana	2.5	8	20.0
3	Mango	3.5	12	42.0
4	Watermelon	2.0	20	40.0
5	Grapefruit	3.5	5	17.5
6	Peach	5.0	7	35.0

(13) 'Fruits price.csv'文件已保存到当前目录

(14) 读取的Excel文件前5行数据:

	水果	价格	销量	总金额
0	Apple	3.0	10	30.0
1	Orange	2.0	15	30.0
2	Banana	2.5	8	20.0
3	Mango	3.5	12	42.0
4	Watermelon	2.0	20	40.0